DIGITAL EGYPT PIONEERS INITIATIVE (DEPI) • DATA ENGINEERING TRACK

Project Proposal: Analysis & Design Phase Blueprint

Enterprise Healthcare Clinical, Surgical & Revenue Data Engineering Pipeline — **SURGIQ**

Project Name: SURGIQ (Hospital Data Lakehouse & Warehouse)	**Supervisor / Instructor:** Eng. Alberto Hana
Engineering Team: Aya Elsaify, Yousef Zahran, Ahmed Mokhtar, Ahmed Taha	**Project Timeline:** 17 July 2026 – 10 December 2026
Core Tech Stack: Apache Airflow, Apache PySpark, PostgreSQL, Docker, DBeaver, GitHub, Power BI (Medallion Architecture)	

This document details the structured components required for the Analysis and Design sections of the **SURGIQ** graduation project proposal for the DEPI Administration, engineered to process an enterprise-scale dataset of **1,000,000 patient encounters across 103 clinical, operational, and financial attributes**.

0. Foundational DEPI Documentation Components (Completed Milestones)

In alignment with the official **DEPI Project Documentation Guidelines**, the following foundational deliverables establish the project management structure, academic/industry literature benchmarks, and functional/non-functional requirements prior to the System Analysis & Design phase.

0.1 Project Planning & Management (DEPI Industry: 2 Oct 2026 | DEPI: 16 Oct 2026)

The SURGIQ project spans from **17 July 2026 to 10 December 2026** under the supervision of **Eng. Alberto Hana**. Work is distributed across the four data engineers using Agile two-week sprints tracked on GitHub:

- **Aya Elsaify (Orchestration & Containerization Lead):** Manages Docker Compose multi-container orchestration, Apache Airflow DAG scheduling (configured with `LocalExecutor` for container efficiency), watermark-based incremental orchestration, SLA monitoring, and automated retry logic.
- **Yousef Zahran (PySpark Bronze & Silver ETL Lead):** Engineers distributed PySpark ingestion scripts for the 1,000,000-row dataset, ISO-8601 timestamp parsing, SHA-256 PHI cryptographic masking, and automated data quality quarantine routing (`silver_quarantine_encounters`).
- **Ahmed Mokhtar (Gold Warehouse & Dimensional Modeling Lead):** Designs the **15-table Fact Constellation (Galaxy Schema)** in PostgreSQL via DBeaver, isolates sparse satellite entities (`safety_incident`) and periodic inventory snapshots (`surgical_kit_stock_snapshot`), and configures high-throughput bulk loading and B-tree indexing.

- **Ahmed Taha (BI Analytics & Data Governance Lead):** Develops role-based PostgreSQL views, semi-additive DAX measures (`LASTNONBLANK` for kit stock snapshots), and interactive Power BI dashboards covering Clinical ICU/OR Throughput, Revenue Cycle Claims Auditing, and Surgical Supply Chain inventory.

0.2 Literature Review (DEPI Industry: 9 Oct 2026 | DEPI: 16 Oct 2026)

A review of enterprise healthcare data engineering architectures establishes three core paradigms adopted by SURGIQ:

- **Medallion Lakehouse Architecture in Healthcare:** Industry benchmarks demonstrate that decoupling raw clinical ingestion (Bronze) from validated Parquet entities (Silver) and relational warehouse schemas (Gold) prevents data swamp degradation and ensures full auditability when processing high-dimensional Electronic Health Record (EHR) exports.
- **Kimball Fact Constellation (Galaxy Schema) & Mixed-Grain Modeling:** Healthcare analytics literature confirms that wide flat tables (such as our 103-column source schema) mix incompatible analytical grains—combining transactional encounter events with daily supply inventory snapshots and sparse safety incidents. Decomposing wide records into a 15-table Fact Constellation sharing conformed dimensions (`date_dim` , `surgical_kit` , `hospital_unit`) eliminates First Normal Form (1NF) redundancy and prevents fan-trap metric inflation.
- **Automated Healthcare Data Quality & Revenue Integrity:** Clinical informatics studies emphasize isolating structurally sparse events (e.g., safety incidents that occur in only 2% of encounters) into dedicated satellite tables rather than storing 98% null columns, alongside automated rule gates for physiological bounds, chronological ordering, and upcoding/unbundling fraud detection.

0.3 Requirements Gathering (DEPI Industry: 9 Oct 2026 | DEPI: 16 Oct 2026)

Requirement Category	Verified Specification for SURGIQ (1,000,000 Rows × 103 Columns)
Functional Requirements (FR)	• **FR1 (Idempotent Batch Ingestion):** Ingest 1,000,000 patient encounter rows across 103 attributes into partitioned Bronze Parquet storage via Apache Airflow using watermark-based incremental tracking. • **FR2 (Silver Cleansing, PHI Masking & Quarantine):** Split ISO-8601 timestamps into date/clock components, apply salted SHA-256 hashing to sensitive identifiers, and route chronologically or physiologically invalid rows to a dead-letter quarantine table. • **FR3 (15-Table Gold Galaxy Schema):** Normalize cleansed records into 8 Fact/Bridge/Satellite tables and 7 Conformed Dimension tables inside PostgreSQL managed via DBeaver. • **FR4 (Executive BI & Semi-Additive Analytics):** Deliver Power BI dashboards utilizing semi-additive DAX measures for daily surgical kit snapshots and averaged nurse burnout metrics.

Requirement Category	Verified Specification for SURGIQ (1,000,000 Rows × 103 Columns)
Non-Functional Requirements (NFR)	• **NFR1 (Throughput & Query SLA):** Complete end-to-end Bronze-to-Gold batch execution for 1M rows in **under 10 minutes** using Snappy Parquet intermediates and PostgreSQL bulk loading, achieving **sub-2-second Power BI query response times**. • **NFR2 (Container Footprint & Reproducibility):** Constrain Docker Compose memory usage (Airflow `LocalExecutor`, `spark.driver.memory=2g`, `spark.sql.shuffle.partitions=16`) to run reliably on 16GB RAM workstations. • **NFR3 (Idempotency & Data Governance):** Guarantee 100% primary-key deduplication across pipeline re-runs via upsert/partition-overwrite semantics and enforce HIPAA-aligned role-based PostgreSQL views.

1. Project Analysis Component

The Analysis section diagnoses the current hospital data environment, details the structural problem scope across clinical and revenue domains, verifies technical and operational constraints, and assesses implementation risks.

1.1 Problem Statement & Needs Assessment

Establishes the gap between the current hospital data state and desired analytical performance. Requirements are categorized into felt needs (stakeholder requests) and normative needs (regulatory and data engineering standards).

> **PROJECT IMPLEMENTATION: SURGIQ HEALTHCARE PIPELINE**
>
> **Problem Statement:** Hospital surgical, ICU, revenue cycle, and supply chain records across **1,000,000 patient encounters** are ingested as a heavily denormalized, unvalidated wide dataset (`01_bronze_encounters` containing **103 columns**). Because records remain trapped in raw flat files, hospital leadership cannot systematically track critical operational inefficiencies revealed in baseline profiling: an **81.4% average bed occupancy rate** (peaking at 98.4% in critical care units), a **26% 30-day patient readmission rate**, an **18% insurance claim denial rate** (with an 8% upcoding flag rate), and a **30% surgical kit stockout risk rate**. Structurally, the raw wide table mixes incompatible analytical grains—embedding daily surgical kit inventory snapshots and nurse shift burnout scores directly inside patient encounter rows (causing severe fan-trap inflation if summed naively)—while exhibiting 98% sparsity in safety incident attributes, unparsed ISO-8601 timestamps, and First Normal Form (1NF) redundancy.
>
> **Needs Assessment:**

- **Felt Needs (Stakeholder Requests):** Automated, idempotent daily batch ingestion of 1M+ patient rows, grain-accurate separation of clinical encounters from daily surgical kit stock snapshots (`surgical_kit_stock_snapshot`), calculation of surgical turnaround and bed-boarding latencies, and a 15-table PostgreSQL warehouse feeding Power BI dashboards.
- **Normative Needs (Industry & Regulatory Standards):** Strict schema enforcement, composite primary/foreign key referential integrity, dead-letter quarantine routing for corrupted records, and HIPAA-compliant cryptographic masking (SHA-256) of Protected Health Information (PHI) across Medallion layers.

1.2 Stakeholder Analysis

Maps individuals and institutional groups affected by the SURGIQ pipeline, detailing their level of influence and core technical or operational expectations.

Stakeholder Group	Impact / Power	Core Interests & Project Expectations
DEPI Administration & Instructor (Eng. Alberto Hana)	High / High	On-time milestone delivery matching DEPI documentation deadlines, reproducible containerized architecture (Docker/Airflow/PySpark), clean GitHub collaboration, and scalable Star Schema modeling.
Hospital Executive & ICU/OR Administration	High / High	Visibility into ICU/SICU bed occupancy (81.4% baseline), OR schedule variance (planned vs. actual minutes), 30-day readmissions (26%), and Length of Stay (3.28 days average).
Revenue Cycle & Billing Auditors	High / Medium	Automated detection of claim denials (18% baseline across Medicare, BlueCross, Aetna) and billing compliance flags (upcoding and unbundling risk monitoring).
Surgical Supply Chain & Staffing Managers	Medium / Medium	Preventing surgical kit stockouts (30% risk rate across high-cost kits averaging $2,579) and monitoring nurse-to-patient ratios (5.44 average) against staff burnout scores.
SURGIQ Data Engineering Team	High / Medium	Modular PySpark ETL scripts, memory-tuned Docker/Airflow orchestration, grain-accurate 15-table PostgreSQL ERD deployment via DBeaver, and zero merge conflicts on GitHub.

1.3 Feasibility Study

Verifies whether the 1-million-row SURGIQ data pipeline can realistically be executed within technical, organizational, and economic limits using open-source infrastructure.

- **Technical Feasibility:** Processing **1,000,000 rows × 103 columns** (~850 MB–1.2 GB raw CSV) exceeds single-threaded script limits, and naive row-by-row Spark JDBC inserts into PostgreSQL would trigger severe transaction bottlenecks. SURGIQ resolves this by persisting Bronze and Silver layers as **Snappy-compressed, partitioned Parquet files** on disk and loading only the normalized 15-table Gold schema into **PostgreSQL** (managed via **DBeaver**) using partitioned bulk JDBC/ `COPY` operations (`batchsize=10000`).
- **Operational Feasibility:** To prevent container out-of-memory (OOM) crashes on team workstations (8GB–16GB RAM), **Docker Compose** is configured with Airflow's lightweight `LocalExecutor` (eliminating Redis/Celery overhead) and tuned Spark parameters (`spark.driver.memory=2g` , `spark.executor.memory=2g` , `spark.sql.shuffle.partitions=16`). All four engineers collaborate via a shared **GitHub** repository with modular branch workflows.
- **Economic Feasibility:** By engineering the entire stack on **100% open-source software** (Docker, Apache Airflow, Apache PySpark, PostgreSQL, DBeaver, GitHub) paired with academic/desktop **Power BI**, total software licensing and cloud infrastructure expenditure is **$0.00** while delivering enterprise-grade throughput.

2. Project Design Component

The Design section translates the verified operational analysis of the 103-column dataset into concrete architectural specifications, Medallion layer transformations, the 15-table DBeaver ERD schema, DEPI milestone schedules, and risk mitigations.

2.1 Project Objectives & Scope Constraints

Establishes definitive, measurable boundaries around the SURGIQ data engineering execution.

> **SCOPE DEFINITION, ENGINEERING SLAS & PRIMARY FOCUS**
>
> **SMART Objective:** Design, containerize, and automate an idempotent Medallion Architecture data pipeline (Bronze, Silver, Gold) using Docker, Apache Airflow, PySpark, and PostgreSQL that processes and validates **1,000,000 raw hospital encounter records (103 attributes)** into a **15-table Gold Fact Constellation Schema** in **under 10 minutes of batch execution time**, achieving **100% primary-key deduplication**, **zero unhandled schema exceptions** via dead-letter quarantine routing, and **sub-2-second Power BI query response times** by **10 December 2026**.
>
> **Primary Business Domains Selected for Data Engineering Focus:**
>
> 1. **Clinical & Surgical Throughput Operations:** Modeling core patient admissions (`patient_encounter`), multi-procedure surgical logs (`encounter_procedure`), equipment utilization (`encounter_equipment_use`), and isolated safety incidents (`safety_incident`) to monitor bed occupancy, OR duration variances, LOS, and 30-day readmissions.

2. **Healthcare Revenue Cycle & Claims Integrity:** Auditing financial performance in `insurance_claim` across payers (`insurance_payer`), tracking `submitted_charge_usd` , `allowed_amount_usd` , `patient_responsibility_usd` , and isolating denials (`is_denied` , `denial_reason_code`) and compliance flags (`is_upcoding_suspected` , `is_unbundling_suspected`).
3. **Surgical Supply Chain Snapshots & Workforce Capacity:** Decoupling daily kit inventory into a periodic snapshot table (`surgical_kit_stock_snapshot`) to prevent stockout risks across `surgical_kit` SKUs, while tracking shift-level nursing workload and MBI burnout in `encounter_nurse_shift` .

Scope Boundaries:

- **In-Scope:** Memory-tuned Docker Compose deployment; Airflow watermark DAG scheduling; PySpark Bronze-to-Silver cleansing, SHA-256 PHI hashing, and quarantine routing; Silver-to-Gold normalization into the 15 DBeaver PostgreSQL tables; B-tree indexing; Power BI semi-additive DAX dashboards.
- **Out-of-Scope:** Real-time HL7/FHIR bedside hardware streaming, live write-back to production hospital EHR software, and clinical diagnostic machine learning training.

2.2 Technical & System Architecture (Medallion & 15-Table Galaxy Blueprint)

Defines how the 1-million-row dataset flows through the containerized open-source pipeline from raw ingestion to the 15-table PostgreSQL relational warehouse modeled in DBeaver.

Architecture Layer	Core Technologies	Data Engineering Operations & Exact DBeaver Schema Specification
1. Container & Orchestration Layer	Docker Compose, Apache Airflow (`LocalExecutor`), GitHub	Provisions resource-capped containers for Airflow, Spark, and PostgreSQL. Airflow DAGs execute incremental watermark loads (filtering on `admit_timestamp`) with dynamic partition overwrite to guarantee idempotency on task retries.
2. Bronze Layer (Raw Immutable Storage)	Apache PySpark, Snappy Parquet Files	Ingests raw 1,000,000-row batches (`01_bronze_encounters.csv` , 103 columns) directly into partitioned **Bronze Parquet storage** on disk (avoiding raw PostgreSQL I/O bottlenecks). Appends audit metadata (`ingestion_timestamp` , `batch_id` , `source_file`).

Architecture Layer	Core Technologies	Data Engineering Operations & Exact DBeaver Schema Specification
3. Silver Layer (Cleansed, Masked & Quarantined)	Apache PySpark DataFrame API, Parquet Storage	• **Timestamp Decomposition:** Parses 8 ISO-8601 timestamps and splits admission/discharge into explicit date keys (`admit_date` , `discharge_date`) and time attributes (`admit_clock_time` , `discharge_clock_time`) to link cleanly with `date_dim` . • **HIPAA PHI Cryptographic Governance:** Applies salted SHA-256 hashing (`F.sha2()`) and surrogate key generation to sensitive identifiers before downstream exposure. • **Automated DQ Gate & Quarantine Table:** Enforces chronological rules (`triage_time <= admit_date <= discharge_date` , `or_start_time <= or_end_time`), financial rules (`allowed_amount_usd <= submitted_charge_usd`), and physiological bounds (`0 < oxygen_saturation_pct <= 100` , `10 <= bmi <= 80`). Invalid rows route to `silver_quarantine_encounters` with a `dq_failure_reason` tag. • **Snapshot Grain Deduplication:** Derives `snapshot_date` for surgical kits and deduplicates on `(kit_id, snapshot_date)` to prevent primary key collisions.

Architecture Layer	Core Technologies	Data Engineering Operations & Exact DBeaver Schema Specification
4. Gold Layer (15-Table Fact Constellation Warehouse)	PostgreSQL, DBeaver, PySpark Bulk JDBC / `COPY`	Loads validated Silver Parquet entities into PostgreSQL using batched writes (`batchsize=10000`) across our **15-table DBeaver ERD**: • **Core & Periodic Fact Tables (2):** `patient_encounter` (PK: `encounter_id` ; FKs: `patient_id` , `icd10_code` , `assigned_unit_id` , `kit_id` , `admit_date` , `discharge_date`) and `surgical_kit_stock_snapshot` (Composite PK: `kit_id` , `snapshot_date` — periodic inventory grain capturing `current_stock` , `days_of_supply` , `stockout_risk_flag` , `projected_demand_4wk`). • **Bridge & Sub-Encounter Fact Tables (3):** `encounter_procedure` (PK: `encounter_id` , `cpt_code` ; FK: `surgeon_id`), `encounter_equipment_use` (PK: `encounter_id` , `equipment_type_id`), and `encounter_nurse_shift` (PK: `encounter_id` , `nurse_id` — shift hours, nurse ratio, MBI burnout, turnover risk). • **1:1 & Filtered Satellite Tables (3):** `insurance_claim` (PK: `claim_id` ; FKs: `encounter_id` , `payer_id`), `patient_satisfaction_survey` (PK/FK: `encounter_id` — 8 CAHPS scores), and `safety_incident` (PK: `incident_id` ; FK: `encounter_id` — populated *only* when an incident occurs, eliminating 98% null sparsity). • **Conformed Dimension Tables (7):** `date_dim` (PK: `date_id`), `patient` (PK: `patient_id`), `diagnosis` (PK: `icd10_code`), `hospital_unit` (PK: `unit_id`), `surgical_kit` (PK: `kit_id`), `medical_procedure` (PK: `cpt_code`), `surgeon` (PK: `surgeon_id`), `nurse` (PK: `nurse_id` ; FK: `home_unit_id`), `equipment_type` (PK: `equipment_type_id`), and `insurance_payer` (PK: `payer_id`). *(15 total relational tables).*
5. BI & Consumption Layer	Power BI, Role-Based SQL Views	Connects to PostgreSQL Gold tables via role-based views. Implements **semi-additive DAX measures** (`LASTNONBLANK` over `date_dim[date_id]`) for `surgical_kit_stock_snapshot` so inventory is never summed across dates, and uses `AVERAGE()` for nurse burnout metrics.

2.3 Timeline, DEPI Documentation Schedule & Risk Matrix

Outlines the official **Project Documentation Guidelines** schedule mandated by DEPI Industry and DEPI Administration (within our overall project timeframe from **17 July 2026 to 10 December 2026**), followed by technical risk mitigations.

Item	DEPI Industry	DEPI	SURGIQ Deliverable & Execution Status
Project Planning & Management	**2 Oct 2026**	**16 Oct 2026**	Project charter (kickoff 17 Jul 2026), team role allocation (Aya, Yousef, Ahmed Mokhtar, Ahmed Taha), GitHub & Docker setup.
Literature Review	**9 Oct 2026**	**16 Oct 2026**	Benchmarking Medallion Lakehouse architectures, Kimball Fact Constellation schemas, and clinical/claims auditing standards.
Requirements Gathering	**9 Oct 2026**	**16 Oct 2026**	Profiling `01_bronze_encounters.csv` (1M rows × 103 columns), functional/non-functional SLAs, and KPI definitions.
System Analysis & Design	**9 Oct 2026**	**6 Nov 2026**	Complete Analysis & Design Blueprint (this report), Bronze-Silver-Gold PySpark specs, and 15-table DBeaver ERD modeling.
Implementation (Source Code & Execution)	**16 Oct 2026**	**30 Nov 2026**	Coding Airflow DAGs and PySpark ETL jobs, executing 1M-row transformations, loading 15 PostgreSQL Gold tables, and Power BI DAX.
Final Presentation & Testing & Reports	**16 Oct 2026**	**4 Dec 2026**	End-to-end pipeline QA testing, quarantine audit verification, final report submission (4 Dec), and project wrap-up (10 Dec 2026).

Identified Risk Event	Probability	Designed Mitigation Strategy
Docker Container OOM Crashes on Local Laptops	Medium	Configure Airflow with `LocalExecutor` (removing Redis/Celery containers) and cap PySpark memory in `docker-compose.yml` (`spark.driver.memory=2g` , `spark.sql.shuffle.partitions=16`).
PostgreSQL JDBC Write Bottlenecks on 1M Rows	High	Keep Bronze and Silver layers as Snappy-compressed Parquet files on storage; load only the normalized 15 Gold tables into PostgreSQL using tuned JDBC batch writes (`batchsize=10000`) or native `COPY` , building B-tree indexes post-load.
Grain Distortion & Fan-Trap Sums on Kit Stock	Medium	Isolate daily kit metrics into `surgical_kit_stock_snapshot` keyed by `(kit_id, snapshot_date)` via PySpark deduplication, and enforce semi-additive `LASTNONBLANK()` DAX formulas in Power BI.
Corrupted Records or Duplicate Re-Run Loads	Medium	Route chronological, financial, or physiological outliers to `silver_quarantine_encounters` without halting the DAG, and enforce idempotent watermark/upsert loads per batch.